

Recurrent Neural Network.(RNN)

* Feed forward Neural Network *

Problem : "Nepal is a country" when the word come second or third time it actually cannot understand its a country.

Consequence : This require massive parameter to relearn the rule.

cannot handle sequences of varying lengths.

* RNN *

parameter sharing.

have same weight for each position
meaning (General rule like adjective precedes noun

* RNN vs 1D-convolution *

1D convolution : Shallow sharing

As we know, convolution also share parameter but they are shallow in time dimension.

Output at time 't' calculated by small, fixed window of neighboring input

Recurrent Neural Network

↳ its output is the function of previous output

↓
state at time t depend on $t-1$ which
depend on $t-2$

Note :- A deep computational graph, while convolution only look for the local snapshot.

RNN retain memory of entire history of the sequence up to that point

Recurrent Equation

$$s^{(t)} = f(s^{(t-1)}, \theta)$$

where,

$s^{(t)}$ = state of the system at time t

θ = The set of parameter (weight and biases)

θ is constant doesnot change with time ' t '

This equation is recurrent because My current state is determine by previous state.

The concept of unfolding.

To train RNN using backpropagation, we must transfer cycle to chain.

A cycle in a graph represent the influence of variable on itself in the future.

Example

$$T = \infty$$

At $T=1$

$$s^{(1)} = f(s^{(0)}; \theta)$$

At $T=2$

$$s^{(2)} = f(s^{(1)}; \theta)$$

substitute $s^{(1)}$

$$s^{(2)} = f(f(s^{(0)}; \theta), \theta)$$

At $T=3$

$$s^{(3)} = f(s^{(2)}; \theta)$$

$$= f(f(f(s^{(0)}; \theta); \theta); \theta)$$

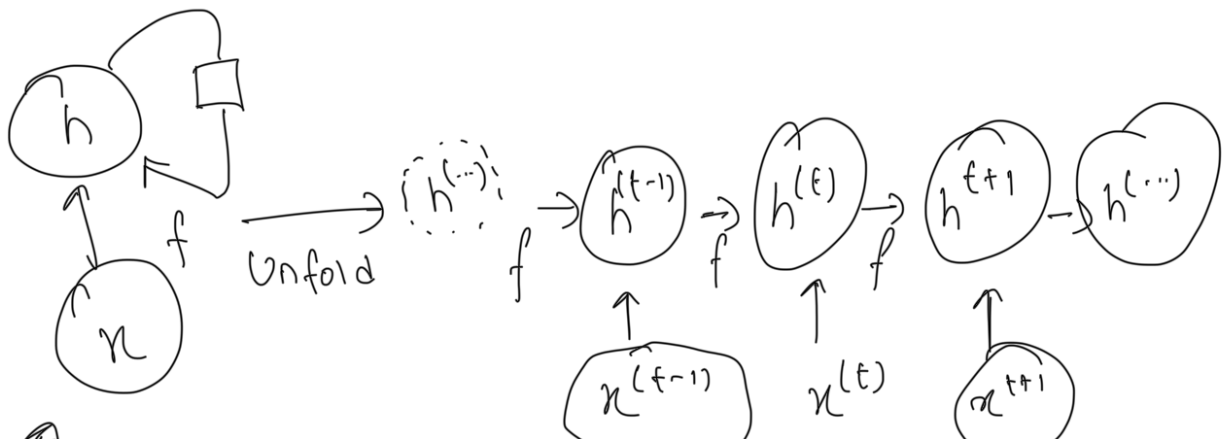
Why?

Remove of Recurrence :->

Just deep function of the
initial state $s^{(0)}$

Standard Graph :->

- Directed Acyclic Graph
- Clear and linear



✓
Cyclic

Equations :

State update :

$$h_t = f(h_{t-1}, x_t)$$

Activation function :

$$h_t = \tanh(\omega_{hh} \cdot h_{t-1} + \omega_{nh} \cdot x_t)$$

ω_{hh} weight matrix for the recurrent neuron

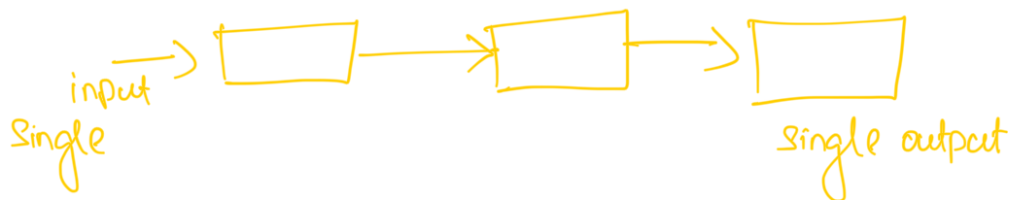
ω_{nh} weight matrix for the input neuron

Output :

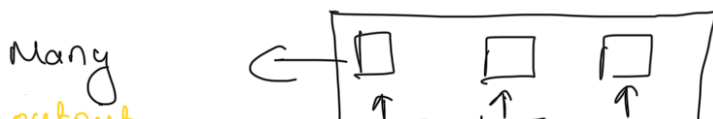
$$y_t = \omega_{hy} \cdot h_t$$

Types :

① one to one RNN

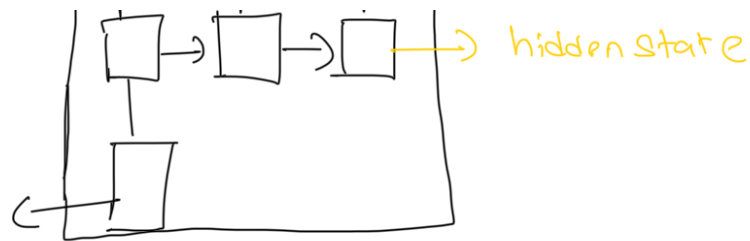


② one to many

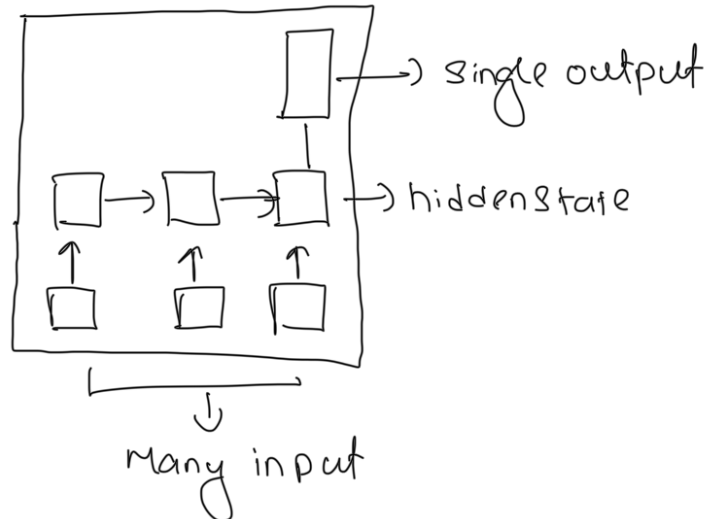


output

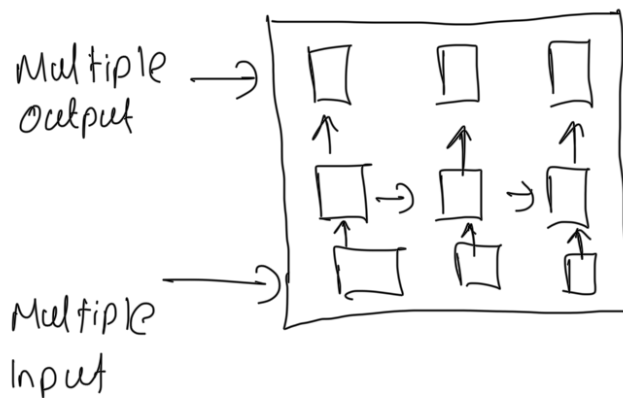
Single
input



③ Many to one



④ Many to many



Variant of RNN

① Vanilla RNN

② Bidirectional RNN

③ Long short term Memory (LSTM)

④ Gated Recurrent Units (GRU)

